

Rapport de Stage — Orbyte

Rapport de Stage — Système RAG Orbyte

Établissement d'accueil : SESAME Innovation Lab

Encadrant : Dr. Amine OTHMAN

Période du stage : Été 2026

Équipe stagiaires : Asma, Ahmed, Chayma, Mohamed, Rayane, Aziz

1. Introduction et contexte

Ce stage s'inscrit dans le cadre du projet Orbyte, une plateforme d'intelligence artificielle conversationnelle développée au sein du laboratoire SESAME dans le cadre du Challenge Projets d'Entreprendre. Le projet part d'un constat simple : les employés perdent chaque jour un temps considérable à rechercher des informations déjà existantes au sein de leur entreprise, faute d'un point d'accès unique à la documentation interne. Orbyte propose d'y répondre par un agent IA dédié à la connaissance interne de chaque entreprise, capable de centraliser ses documents et d'y répondre de façon conversationnelle, fiable et entièrement sécurisée en local.

Un premier prototype avait été développé en amont du stage. L'audit initial de ce prototype a mis en évidence des limites structurelles rédhibitoires, conduisant à la décision de reconstruire le système intégralement depuis zéro.

2. Diagnostic initial : limites du prototype Orbyte v1

L'audit du premier prototype a identifié quatre catégories de problèmes :

1. **Chunking fragile** : le découpage des documents reposait sur des séparateurs de chaînes de caractères fixes, sans prise en compte de la nature sémantique du contenu ni du type de section (texte libre, tableau, image).
 2. **Recherche dense uniquement** : le prototype n'effectuait que de la recherche vectorielle (dense), sans recherche par mots-clés, rendant les requêtes précises ou factuelles peu efficaces.
 3. **Filtrage des permissions peu fiable** : la gestion des droits d'accès aux documents n'était pas intégrée au pipeline de retrieval, exposant potentiellement des documents confidentiels.
 4. **Architecture éclatée entre trois stacks** : la logique métier était distribuée de manière incohérente entre des composants Java et Python, rendant la maintenance et l'évolution du système très complexes.
-

3. Décisions d'architecture : pourquoi une reconstruction complète

Face à ces limites, la décision de reconstruction complète a été justifiée par : - L'impossibilité de corriger le chunking et le retrieval sans réécrire l'intégralité du pipeline d'ingestion. - La nécessité d'unifier le backend en un seul langage (Python) pour éliminer la dette technique et la duplication. - L'adoption d'un modèle d'indexation hybride (vectoriel + mots-clés) supporté nativement par les moteurs de recherche modernes. - La volonté d'intégrer dès la conception un modèle de permissions robuste, appliqué directement au niveau du moteur de recherche.

4. Travail réalisé

4.1 Architecture générale

Confirmation de l'unification backend en Python. L'intégralité du backend a été refondue en Python 3.13. Toute logique redondante issue d'anciennes piles a été supprimée au profit d'une stack moderne et homogène :

- **FastAPI** : serveur d'API REST haute performance (point d'entrée principal de l'application, orchestré via Uvicorn en mode asynchrone).
- **SQLAlchemy + Alembic** : couche d'accès aux données relationnelles (ORM) et système de migrations incrémentales de schéma.
- **Celery** : moteur d'exécution de tâches asynchrones en arrière-plan, gérant les files d'attente distribuées.
- **Supervisord** : gestionnaire et superviseur multi-processus au sein des conteneurs d'application.

Organisation modulaire du backend :

Module / Sous-système	Rôle technique
Ingestion & Indexation	Pipeline de chunking multimodal, calcul d'embeddings et insertion vectorielle
Moteur d'indexation	Abstraction unifiée et connecteurs vers OpenSearch et Vespa
Recherche & Retrieval	Pipeline de recherche hybride (dense + creuse), filtres d'accès ACL et réordonnancement (reranking)
Traitement asynchrone	Gestion des workers Celery, ordonnanceur de tâches périodiques et gestion des files d'attente
Authentification	Gestion des utilisateurs, tokens JWT, OAuth2, clés d'API et contrôle de session
Sécurité & DLP	Détection de fuites de données sensibles (Data Loss Prevention) et audit
Outils agentiques	Contrat d'interface et moteur d'exécution d'outils pour agents IA
Deep Research	Agent autonome de recherche approfondie avec boucle multi-étapes
Coding Agent	Agent de génération et d'exécution sécurisée de code
Connecteurs	Passerelles d'extraction vers les sources documentaires tierces
Modèles relationnels	Schémas relationnels et persistance PostgreSQL
Fournisseurs LLM	Abstraction unifiée des modèles de langage (locaux et distants)
Bibliothèque de prompts	Centralisation et paramétrage des invites système (<i>system prompts</i>)

4.2 Pipeline d'ingestion et chunking multimodal

Stratégie de découpage sémantique adaptée à chaque nature de contenu :

Le système met en œuvre un orchestrateur de chunking qui aiguille chaque section du document vers un découpeur spécialisé selon la typologie détectée :

- **Découpage textuel (`TextChunker`)** : s'appuie sur la bibliothèque spécialisée **Chonkie** (`SentenceChunker`) pour opérer un découpage respectueux de la syntaxe et des limites de phrases. Il gère dynamiquement l'accumulation en mémoire tampon selon le budget de tokens, la segmentation contrôlée des passages denses et la continuité contextuelle entre blocs.
- **Découpage visuel (`ImageChunker`)** : chaque image forme une unité documentaire autonome référencée par un identifiant unique, avec la capacité d'ajouter une description sémantique générée par un modèle de vision multimodal.
- **Découpage tabulaire (`TabularChunker`)** : dédié aux feuilles de calcul et formats CSV. Chaque ligne est modélisée sous format clé-valeur (`en-tête=valeur`), enrichie par un algorithme d'analyse structurelle préalable qui conserve la signification des métadonnées du tableau.

Paramètres clés du pipeline d'ingestion :

Paramètre	Valeur	Rôle
Limite de tokens par chunk	Calibrée sur la fenêtre de contexte du modèle d'embedding	Évite la troncature d'information lors du calcul vectoriel
Chevauchement (<i>Overlap</i>)	0 (aucun chevauchement)	Choix architectural explicite évitant la redondance d'indexation
Taille de l'extrait (<i>Blurb</i>)	128 tokens	Résumé rapide pour l'affichage synthétique des résultats
Part maximale de métadonnées	25 %	Garantit qu'au moins 75 % du chunk est constitué de contenu textuel utile
Seuil minimal de contenu	256 tokens	Élimine les micro-fragments non significatifs
Profondeur de réordonnancement	1 000 candidats	Volume de documents pré-sélectionnés soumis au reranker

Le pipeline intègre également une option de **Contextual RAG** permettant de générer automatiquement des résumés contextuels par LLM en amont de l'indexation.

4.3 Recherche hybride et réordonnancement (Reranking)

Recherche hybride équilibrée. Pour pallier les angles morts de la recherche purement vectorielle, le système combine systématiquement la recherche dense (sémantique) et la recherche creuse (BM25 par mots-clés) via un coefficient d'équilibrage :

```
HYBRID_ALPHA = max(0, min(1, float(os.environ.get("HYBRID_ALPHA") or 0.5)))  
# 0 = recherche exclusive par mots-clés, 1 = recherche purement vectorielle,  
  0.5 = équilibre
```

Le moteur de recherche orchestre la requête en extrayant simultanément les vecteurs d'embedding et les termes significatifs (après élimination des mots vides via NLTK). Lorsque

la recherche est sollicitée par un agent autonome, la fusion des listes de résultats s'effectue via l'algorithme **RRF** (*Reciprocal Rank Fusion*, constante $k = 50$).

Réordonnement sémantique (Reranking). Une étape de reranking affine les documents pré-sélectionnés avant transmission au LLM. Le composant supporte : - L'intégration de modèles distants spécialisés (API Cohere, AWS Bedrock). - L'utilisation de passerelles unifiées (LiteLLM) pour les modèles hébergés ou tiers. - L'interrogation d'un serveur d'inférence local dédié à haute performance.

4.4 Infrastructure et conteneurisation

L'ensemble des services nécessaires au fonctionnement du système est orchestré via une flotte de conteneurs Docker :

Service	Rôle opérationnel
Serveur d'API	Exécution de l'application FastAPI, endpoints REST et contrôle d'accès
Workers d'arrière-plan	Exécution asynchrone supervisée des pipelines d'ingestion et de maintenance
Interface Web	Application moderne Next.js pour les interactions utilisateurs
Serveur d'inférence	Service dédié au calcul local des embeddings et des opérations de reranking
Base relationnelle	Instance PostgreSQL dédiée au stockage des métadonnées, sessions et configurations
Moteur de recherche	Cluster OpenSearch pour l'indexation hybride (vecteurs k-NN et BM25)
Mémoire cache & Broker	Instance Redis pour la distribution des tâches Celery et le cache applicatif
Reverse Proxy	Nginx gérant le routage, la terminaison réseau et la sécurité des ports
Stockage d'objets	Instance MinIO (compatible protocole S3) pour la persistance des fichiers bruts
Sandbox d'exécution	Environnement isolé d'exécution de code Python pour les agents

Exploitation des ressources de calcul locales : Le modèle de génération principal (Qwen 2.5 14B) est servi directement sur l'environnement hôte via Ollama, permettant au système d'exploiter la totalité des capacités d'accélération matérielle (GPU) sans surcoût de virtualisation.

4.5 Traitement asynchrone et planification

Afin de garantir une interface utilisateur réactive lors de l'ingestion de volumineux corpus documentaires, tous les traitements lourds sont délégués à des workers asynchrones

organisés en files spécialisées :

- **Coordinateur principal** : analyse périodiquement l'état des connecteurs documentaires.
- **Workers légers** : dédiés aux opérations rapides à haute concurrence (synchronisation de statuts, suppressions).
- **Workers intensifs** : affectés aux calculs longs à concurrence restreinte (génération de rapports, purges).
- **Workers d'ingestion documentaire** : dédiés au chunking, à l'inférence d'embedding et à l'écriture vectorielle.
- **Workers de collecte** : gèrent les requêtes vers les API des sources de données externes.
- **Workers de fichiers utilisateurs** : prennent en charge les documents soumis en direct dans le chat.
- **Supervision & Watchdog** : un mécanisme de surveillance automatisé garantit la résilience des ordonnanceurs et redémarre les processus en cas de défaillance.

4.6 Authentification et contrôle d'accès (RBAC / ACL)

Authentification complète : La couche d'authentification s'appuie sur le standard OAuth2 avec jetons JWT signés, gestion des clés API personnelles (PAT), invitations d'utilisateurs, validation des domaines d'adresses et limitation de débit (*rate limiting*).

Permissions à granularité fine : Le modèle de droits repose sur un principe d'implication hiérarchique où les prérogatives supérieures englobent automatiquement les privilèges subordonnés. Des rôles spécifiques ont été formalisés (`AGENT_CREATOR` , `SECURITY_AUDITOR` , `DOCUMENT_MANAGER`).

Filtrage ACL strict à la source : La sécurité des documents repose sur un filtrage appliqué directement au niveau de la requête au moteur de recherche. La liste de contrôle d'accès de l'utilisateur est injectée comme filtre obligatoire dans la requête d'indexation : un utilisateur ne peut ainsi jamais récupérer, même partiellement, un chunk auquel son profil n'a pas explicitement droit. Par défaut, seuls les documents étiquetés publics sont interrogeables.

Protection contre les fuites de données (DLP) : Un sous-système de contrôle analyse les invites des utilisateurs afin d'intercepter les données sensibles (informations bancaires, secrets d'authentification, identifiants administratifs ou mentions confidentielles) avant qu'elles ne soient relayées.

4.7 Création d'agents métier personnalisés (*Personas*)

Orbyte dépasse le cadre d'un simple chatbot généraliste en permettant de créer et de déployer des **agents IA spécialisés** pour des missions de travail concrètes (par exemple : un **agent de synthèse** pour résumer des réunions et extraire les plans d'action, un agent d'onboarding RH ou un assistant d'analyse documentaire).

Chaque agent se configure simplement selon quatre critères : - **Rôle et directives (*Prompts*)** : Définition précise de sa posture, du ton attendu et du livrable souhaité. - **Périmètre documentaire dédié** : Restriction de sa recherche à des dossiers ou documents précis de l'entreprise pour éviter toute pollution hors sujet. - **Outils associés** : Activation sélective des fonctionnalités d'action (recherche interne avec réordonnancement, recherche web, calcul). - **Partage et contrôle d'accès** : L'agent peut rester privé pour son créateur, être partagé avec une équipe ou être publié dans le catalogue d'entreprise (rôle `AGENT_CREATOR`).

4.8 Déploiement et réseau local

Conteneurisation multi-étapes : La construction des conteneurs applicatifs s'appuie sur une séparation stricte entre la phase de compilation (gestion des dépendances avec vérification cryptographique des sommes de contrôle) et la phase d'exécution allégée, exécutée sous utilisateur système restreint sans privilèges d'administration.

Déploiement sur réseau local (LAN) : Un script d'initialisation détecte dynamiquement l'adresse IP de la machine hôte sur le réseau d'établissement et configure automatiquement les domaines autorisés ainsi que les règles de partage de ressources (CORS). Grâce à l'usage de sous-domaines à résolution dynamique, l'ensemble des utilisateurs connectés au réseau local peuvent accéder à la plateforme sans nécessiter d'infrastructure DNS dédiée.

Sécurité de l'exposition réseau : Seuls les ports d'entrée du reverse proxy Nginx sont exposés sur les interfaces réseau. La base de données, la mémoire cache, le stockage objet et les services d'inférence restent confinés dans le réseau virtuel interne de conteneurs.

4.9 Analyse de la couverture de tests

Constat technique — Nécessité de formalisation d'une suite de tests dédiée.

L'analyse de l'infrastructure de tests démontre que le socle de tests présent dans le dépôt est issu du cadre technique générique et ne couvre pas encore spécifiquement les modules personnalisés créés pendant le stage (règles de rôles personnalisées, module de contrôle DLP, particularités du découpage tabulaire et configuration réseau dynamique).

Ce point constitue le **chantier prioritaire** avant tout passage en phase pilote opérationnelle, afin de garantir l'absence de régression lors des montées de version.

5. Bilan d'avancement et synthèse technique

Composants opérationnels et finalisés

Fonctionnalité	Niveau de maturité	Réalisation technique
Architecture backend unifiée	Terminé	Stack 100 % Python, FastAPI, ORM et orchestration asynchrone
Pipeline d'ingestion multimodal	Terminé	Découpage spécialisé adapté aux textes, images et données tabulaires
Recherche hybride	Terminé	Fusion vectorielle et lexicale avec pondération dynamique
Réordonnancement sémantique	Terminé	Reranking multi-fournisseurs (local, distant et passerelle)
Traitement asynchrone	Terminé	Files de travail Celery découplées avec surveillance automatique
Authentification et sessions	Terminé	Gestion des utilisateurs, tokens JWT, clés d'API et contrôle d'accès
Filtrage documentaire ACL	Terminé	Cloisonnement strict des droits au niveau du moteur d'indexation
Agentification & Assistants métier	Terminé	Création d'agents personnalisés (synthèse, support, analyse), prompts dédiés et périmètre documentaire

Fonctionnalité	Niveau de maturité	Réalisation technique
Outillage & Exécution d'actions	Terminé	Attribution sélective d'outils aux agents (recherche interne, web, calcul)
Déploiement multi-conteneurs	Terminé	Orchestration Docker Compose des 11 micro-services
Configuration réseau local	Terminé	Initialisation automatisée et accès multi-postes sur réseau local

Travaux prioritaires avant mise en production

1. Développement de la suite de tests unitaires et d'intégration :

- Validation automatisée des règles de permissions et rôles utilisateurs.
- Tests de robustesse sur l'analyseur de données tabulaires et de documents complexes.
- Validation fonctionnelle des filtres d'expression régulière du module de sécurité DLP.
- Mesures de performance sous charge simulant de 10 à 20 requêtes simultanées.

2. Parachèvement des liaisons applicatives :

- Finalisation du raccordement des rôles personnalisés sur la totalité des routes de l'API.
- Activation systématique du filtre DLP au sein du flux conversationnel.

3. Préparation du déploiement en environnement institutionnel :

- Remplacement systématique des identifiants et clés de chiffrement par défaut par des secrets d'administration générés aléatoirement.
- Activation du protocole HTTPS sécurisé avec renouvellement automatique des certificats.
- Campagne d'essais grandeur nature sur le réseau Wi-Fi de l'établissement SESAME.

6. Conclusion et perspectives

Le travail accompli au cours de ce stage a permis de concevoir une plateforme RAG moderne, modulaire et robuste. La décision initiale de repartir sur une architecture neuve et unifiée en Python s'est avérée judicieuse : elle a permis de supprimer la complexité d'une pile hétérogène, d'implémenter un découpage documentaire réellement adapté aux différents types de contenus et d'intégrer une recherche hybride associée à un réordonnancement sémantique de premier ordre.

Le système dispose désormais d'un socle technique solide, capable d'évoluer vers un déploiement institutionnel complet au sein du laboratoire SESAME dès lors que la couverture de tests et les procédures de sécurisation des secrets auront été finalisées.

Rapport de stage — Projet Orbyte — Septembre 2026.